Лекция 2. Репозиторий.

1.1. **Введение**

Представьте себе студента, который пишет курсовую работу. Сначала у него есть файл «курсовая.docx». Потом он вносит правки и сохраняет как «курсовая_финал.docx». Затем научный руководитель просит что-то исправить, и появляется «курсовая_финал_точно_финал.docx». Потом друг помогает с оформлением, и рождается «курсовая_ исправленная_версия_2.docx». А ещё через неделю выясняется, что лучший вариант был в файле, который студент случайно перезаписал, и теперь непонятно, где какая правка и что вообще изменилось.

Эта ситуация – классический пример того, как отсутствие системы управления версиями превращает работу в хаос. И если для одного документа это ещё терпимо, то представьте, что таких файлов сотни: код, картинки, настройки, тесты, документация. И над проектом работает не один человек, а команда из десяти разработчиков, каждый из которых сидит в своём городе, а то и в другой стране.

1.2. **Проблема**, которую нужно решить

Когда над проектом работает больше одного человека, возникает множество проблем:

- **Конфликты правок.** Два разработчика одновременно изменили один и тот же файл. Чья версия правильная? Как объединить изменения, не потеряв ничего?

- **Потеря истории**. Кто и когда внёс баг? Какую строчку кода изменили неделю назад, из-за которой всё сломалось? Без истории ответить невозможно.

- **Откат изменений**. Новый функционал оказался неудачным, нужно вернуться к состоянию «как было вчера». Как это сделать, если старые копии уже затёрты?

- **Синхронизация**. У одного разработчика на компьютере свежая версия, у другого – устаревшая. Как всем работать с одним и тем же кодом?

- **Резервное копирование**. Жёсткий диск сгорел. Вся работа пропала? Или где-то есть копия?

- **Эксперименты**. Хочется попробовать новую библиотеку, но страшно сломать работающий код. Где «поиграться», чтобы не навредить основной версии?

Все эти проблемы решает система контроля версий (Version Control System, VCS) и репозиторий как её центральное понятие.

1.3. Система контроля версий

Система контроля версий – это программное обеспечение, которое позволяет:

- хранить историю всех изменений файлов;

- возвращаться к любой предыдущей версии;

- работать над проектом параллельно нескольким людям;

- объединять изменения из разных источников;

- отслеживать, кто, когда и зачем внёс ту или иную правку.

Сегодня самой популярной системой контроля версий является Git. Она была создана Линусом Торвальдсом в 2005 году для разработки ядра Linux и с тех пор стала стандартом в индустрии. По данным опросов разработчиков, более 90% программистов используют Git в своей работе.

1.4. **Плюсы использования Git**

Даже если вы пока пишете лабораторные работы в одиночку, навык работы с репозиториями:

- **обязателен при трудоустройстве.** Ни одна серьёзная компания не представляет себе разработчика без знания Git.

- **формирует портфолио.** Ваш профиль на GitHub – это живая демонстрация навыков для работодателя.

- **экономит время**. Больше не нужно хранить «проект_v1», «проект_v2», «проект_final».

- **учит дисциплине**. Структурированный код и понятная история коммитов – признак профессионала.

2. Что такое репозиторий

2.1. Определение репозитория

Репозиторий (repository, «репо») – это хранилище проекта, которое содержит:

- все файлы проекта;

- полную историю их изменений;

- метаданные: кто, когда и что менял;

- информацию о ветках, тегах и настройках.

Простыми словами, репозиторий – это «папка проекта + машина времени + система совместной работы» в одном флаконе.

2.2. Локальный и удалённый репозиторий

Различают два вида репозиториев:

Локальный репозиторий – это папка на вашем компьютере, в которой инициализирован Git. Внутри неё появляется скрытая директория «.git», где хранится вся служебная информация: история коммитов, ветки, настройки. Обычные файлы проекта (код, картинки) лежат рядом, и вы работаете с ними как обычно.

Удалённый репозиторий – это копия проекта, размещённая на сервере (GitHub, GitLab, Bitbucket и т.п.). **Она нужна для:**

- резервного копирования;

- совместной работы (несколько разработчиков синхронизируют свои локальные копии с удалённой);

- **публикации проекта** (open source);

- **автоматической сборки, тестирования и развёртывания.**

Локальный и удалённый репозитории связаны: вы вносите изменения локально, а затем **«отправляете» их на сервер («push»)** или **«забираете» чужие изменения к себе («pull»).**

2.3. Ключевые термины Git

Прежде чем двигаться дальше, разберём основные понятия, без которых работа с репозиторием невозможна.

Коммит (commit) – зафиксированное состояние проекта в определённый момент времени. Каждый коммит имеет уникальный идентификатор (хеш, например «a3f5c9b»), **автора, дату и сообщение с описанием изменений.** Коммит – это «снимок» проекта, к которому всегда можно вернуться.

Ветка (branch) – независимая линия разработки. По умолчанию в Git есть основная ветка (обычно «main» или «master»). Разработчики создают отдельные ветки под задачи. Это позволяет вести параллельную работу, не мешая друг другу.

Слияние (merge) – объединение изменений из одной ветки в другую. Например, когда функционал готов, ветку сливают в «main».

Конфликт (conflict) – ситуация, когда Git не может автоматически объединить изменения (например, две ветки изменили одну и ту же строку по-разному). Разработчик разрешает конфликт вручную.

Тег (tag) – метка на конкретном коммите. Обычно используется для обозначения версий релиза: «v1.0.0», «v2.1.3».

HEAD – указатель на текущий коммит, с которым вы работаете.

Индекс (staging area) – промежуточная область, куда вы добавляете изменения перед коммитом. Команда «git add» помещает файлы в индекс, «git commit» фиксирует их.

Рабочая директория (working directory) – файлы проекта в том виде, в котором вы их сейчас видите и редактируете.

.gitignore – файл, в котором перечислены шаблоны файлов и папок, которые Git должен игнорировать. Туда обычно попадают временные файлы, кэш, папки с зависимостями («node_modules», «venv»), файлы с секретами («.env», ключи API).

2.4. Модель работы Git

Git можно представить как три «дерева»:

1. Рабочая директория – то, что вы редактируете.

2. Индекс (staging area) – подготовленные к коммиту изменения.

3. Репозиторий (.git) – зафиксированные коммиты.

Типичный цикл работы:

Изменил файл → git add → git commit → git push

(рабочая дир.) (индекс) (репозиторий) (удалённый репо)

2.5. Что хранится в репозитории

В хорошем репозитории лежит не только код, но и всё, что помогает проекту жить:

- исходный код приложения;
- тесты;
- документация (README, docs);
- конфигурационные файлы;
- файлы сборки (package.json);
- лицензия;
- файл «.gitignore»;
- история изменений в «.git».

Чего в репозитории быть не должно:

- пароли, токены, ключи API;
- личные данные пользователей;
- тяжёлые бинарные файлы;
- папки с зависимостями (их восстанавливают командой установки);
- кэш и временные файлы;
- файлы IDE (например, «.idea/», «.vscode/», если они не нужны всей команде).

Часть 3. Где можно пользоваться репозиторием

3.1. Что такое GitHub

GitHub – это платформа для хостинга Git-репозиториев. Она была основана в 2008 году, а в 2018-м выкуплена Microsoft. Сегодня GitHub – крупнейший в мире хостинг кода: на нём размещено более 400 миллионов репозиториев, и им пользуются как отдельные разработчики, так и крупнейшие корпорации.

GitHub предоставляет:

- хранение репозиториев;

- веб-интерфейс для просмотра кода, истории, веток;

- систему отслеживания задач и багов;

- pull requests – механизм предложения изменений и код-ревью;

- автоматическую сборку и тестирование.

3.2. Альтернативы GitHub

- GitLab – мощная платформа с открытым исходным кодом. Можно развернуть на своём сервере. Популярна в корпоративной среде.

- SourceForge – старейшая платформа, популярная в эпоху до GitHub.

Для обучения и портфолио GitHub – оптимальный выбор: бесплатный, с огромным сообществом, отличной документацией и интеграцией с множеством инструментов.

3.3. **Что даёт GitHub разработчику**

- **Портфолио**. Ваш профиль показывает, над чем вы работаете, как часто коммитите, как оформляете код.

- **Сотрудничество.**

- **Обучение.** Чтение чужого кода на GitHub – один из лучших способов расти.

- **Автоматизация**.

Часть 4. Создание репозитория и работа с ним через консоль

4.1. Установка и настройка Git

1. Перед началом работы нужно установить Git:

- Windows: скачать установщик с git-scm.com.

- Linux: «sudo apt install git» (Debian/Ubuntu) или «sudo dnf install git» (Fedora).

2. После установки нужно представиться – эти данные будут попадать в каждый коммит:

```
git config --global user.name "Иван Иванов"
git config --global user.email "ivan@example.com"
```

3. Проверить настройки:

```
git config --list
```

4.2. Создание локального репозитория

У вас есть два пути:

Путь 1. Инициализация нового проекта.

```
mkdir my-project
cd my-project
git init
```

В папке появится скрытая директория «.git» – это и есть локальный репозиторий.

Путь 2. Клонирование существующего проекта.

```
git clone https://github.com/username/repo.git
```

Git скачает все файлы и всю историю коммитов.

4.3. Создание удалённого репозитория на GitHub

1. Зарегистрируйтесь на github.com.
2. Нажмите «New repository».
3. Укажите имя («my-project»), описание, выберите публичный или приватный.
4. Можно сразу добавить «README.md» и «.gitignore».
5. Нажмите «Create repository».

После создания GitHub покажет URL репозитория, например:

https://github.com/username/my-project.git

4.4. Связывание локального и удалённого репозиториев

Если репозиторий создан локально, нужно добавить удалённый:

```
git remote add origin https://github.com/username/my-project.git
git branch -M main
git push -u origin main
```

- «git remote add origin ...» – привязывает удалённый репозиторий под именем «origin».
- «git branch -M main» – переименовывает текущую ветку в «main».
- «git push -u origin main» – отправляет коммиты на GitHub и запоминает связь.

4.5. Базовый рабочий цикл

1. Проверить состояние:

```
git status
```

Показывает, какие файлы изменены, какие добавлены в индекс, какие не отслеживаются.

2. Добавить изменения в индекс:

```
git add file.txt      один файл
git add .         все изменения
git add src/        всю папку
```

3. Зафиксировать коммит:

`git commit -m` "Добавил форму авторизации"

Сообщение коммита должно быть осмысленным: не «fix», а «Исправил ошибку валидации email в форме регистрации».

4. Отправить на удалённый репозиторий:

```
git push
```

5. Получить чужие изменения:

```
git pull
```

4.6. Работа с ветками

Создать ветку:

```
git branch feature/login
```

```
git checkout feature/login
```

Или короче:

```
git checkout -b feature/login
```

Переключиться между ветками:

```
git checkout main
```

Слить ветку в основную:

```
git checkout main
```

```
git merge feature/login
```

Удалить ветку после слияния:

```
git branch -d feature/login
```

4.7. Просмотр истории

`git log` полная история

`git log --oneline` краткая история

`git log --graph --oneline` с графом веток

4.8. Откат изменений

`git checkout -- file.txt` отменить изменения в файле

`git reset HEAD file.txt` убрать из индекса

`git revert <commit-hash>` создать новый коммит, отменяющий изменения

4.9. Работа с удалённым репозиторием

`git remote -v` посмотреть удалённые репозитории

`git fetch` скачать изменения, не сливая

`git pull` скачать и слить

`git push origin main` отправить в конкретную ветку

4.10. Pull Request

Pull Request (PR) – это предложение внести изменения в репозиторий. Обычно используется в командной работе:

1. Вы создаёте ветку в своём репозитории.
2. Вносите изменения, коммитите, пушите.
3. На GitHub нажимаете «Compare & pull request».
4. Описываете, что сделано, зачем.
5. Другие разработчики делают код-ревью, оставляют комментарии.
6. После одобрения PR сливается в основную ветку.

Часть 5. Структура проекта

Структура проекта – это организация файлов и папок внутри репозитория. Хорошая структура:

- делает проект понятным для новых участников;

- упрощает навигацию;

- позволяет автоматизировать сборку и тесты;

- снижает риск ошибок;

- облегчает масштабирование.

Плохая структура – когда в корне лежат десятки файлов без логики: «main.py», «test.py», «style.css», «image.jpg», «data.csv», «backup.zip». Такой проект невозможно поддерживать.

Часть 6. Практический пример

6.1. Постановка задачи

Создадим с нуля репозиторий для небольшого Python-проекта «Калькулятор». Пройдём весь путь: от инициализации до публикации на GitHub.

6.2. Шаг 1. Подготовка окружения

```
mkdir calculator

cd calculator

git  init
```

Создаём структуру в VS Code:

calculator/

├── .gitignore

├── README.md

├── src/

│   └── calculator/

│   ├── __init__.py

│   └── core.py

└── tests/

    └── test_core.py

6.3. Шаг 2. Файл .gitignore

```
__pycache__/
*.pyc
.venv/
venv/
.env
.idea/
.vscode/
*.log
dist/
build/
```

6.4. Шаг 3. Код

src/calculator/core.py:

```
def add(a: float, b: float) -> float:
    return a + b
def subtract(a: float, b: float) -> float:
    return a - b
def multiply(a: float, b: float) -> float:
    return a * b
def divide(a: float, b: float) -> float:
    if b == 0:
        raise ValueError("Деление на ноль невозможно")
    return a / b
```

tests/test_core.py:

```
import sys
from pathlib import Path
sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "src"))
import pytest
from calculator.core import add, subtract, multiply, divide
def test_add():
    assert add(2, 3) == 5
```

```
def test_divide_by_zero():
    with pytest.raises(ValueError):
        divide(1, 0)
```

6.5. Шаг 4. README.md

Простой калькулятор на Python.

Запуск тестов

```
pytest
```

6.6. Шаг 5. Первый коммит

```
git add .
git commit -m "Инициализация проекта: структура, код, тесты"
```

6.7. Шаг 6. Публикация на GitHub

Создаём репозиторий на GitHub (без README и .gitignore – они у нас уже есть), затем:

```
git remote add origin https://github.com/username/calculator.git
git branch -M main
git push -u origin main
```

6.8. Шаг 7. Новая функциональность в ветке

Допустим, нужно добавить возведение в степень:

```
git checkout -b feature/power
```

Добавляем в «core.py»:

```
def power(a: float, b: float) -> float:
    return a ** b
```

Коммитим:

```
git add .
git commit -m "Добавил функцию возведения в степень"
git push -u origin feature/power
```

На GitHub создаём Pull Request из «feature/power» в «main». После ревью (пусть в роли ревьюера выступит воображаемый коллега) PR сливается.

6.9. Шаг 8. Слияние и обновление локальной копии

```
git checkout main
git pull
git branch -d feature/power
```

6.10. Шаг 9. Работа с чужим репозиторием

Если хочется внести вклад в чужой проект:

1. На GitHub нажимаем «Fork» – создаётся копия в вашем аккаунте.
2. Клонируем её:

```
git clone https://github.com/your-username/some-project.git
```

3. Создаём ветку, вносим изменения, коммитим, пушим.
4. На GitHub открываем Pull Request в оригинальный репозиторий.

6.11. Шаг 10. Просмотр истории и откат

```
git log --oneline
```

Пример вывода:

```
a1b2c3d Добавил функцию возведения в степень
e4f5g6h Инициализация проекта: структура, код, тесты
```

6.12. Итог практики

Мы прошли полный цикл:

1. Инициализация репозитория.
2. Настройка «.gitignore».
3. Создание структуры проекта.
4. Первый коммит.
5. Публикация на GitHub.
6. Работа с ветками.
7. Pull Request и слияние.
8. Форк и вклад в чужой проект.

Домашнее задание:

1. Создать аккаунт на GitHub.
2. Создать репозиторий для своей лабораторной работы.
3. Настроить «.gitignore», написать «README.md».
4. Сделать минимум 5 осмысленных коммитов.
5. Создать ветку, внести изменения, открыть Pull Request и слить его.
6. Прислать ссылку на репозиторий.